Dark Store Assortment Optimization

Assortment & Space Optimization for Quick Commerce Dark Stores

A system design framework for SKU rationalization under space and capital constraints

---

Table of Contents

1. Reframing the Problem
2. Data Required
3. If the Data Doesn't Exist Yet: Questions to Ask & Fallbacks
4. System Architecture: The Optimization Pipeline
5. The Core Math: Shelf-Space Allocation Problem (SSAP)
6. The Practical Cascade Before Full MIP
7. Handling Substitution & Cannibalization
8. Network Tiering: The Biggest Structural Lever
9. Inventory & Replenishment Policy
10. ML Techniques Mapped to Each Stage
11. Rollout Roadmap: Crawl → Walk → Run
12. KPIs & Feedback Loop
13. Key Assumptions

---

1. Reframing the Problem

This is a well-studied class of problem in retail science — **assortment optimization + shelf-space allocation under space/capital constraints** — compounded by the fact that quick commerce lets you solve it at a **network** level, not just a single-store level.

There are actually three nested problems here, not one:

1. **Assortment selection** — which SKUs deserve a place in *this* store at all (out of 75 hair-oil SKUs, which 15–20 make the cut).
2. **Space/inventory allocation** — of the chosen SKUs, how many facings/how much stock each gets.
3. **Network tiering** — since delivery is fast anyway, does every SKU need to be *in* every dark store, or can slow-movers be served from a regional micro-fulfillment center within the same SLA?

Point 3 is the one people miss most often, and it's the biggest lever available. Quick commerce doesn't have to look like a mini-supermarket. It can look like a **tiered inventory network**:

- **Core SKUs** in every dark store
- **Tier-2 SKUs** in a regional hub feeding 5–10 dark stores
- **Tier-3 (long tail) SKUs** served from a larger fulfillment center with a slightly relaxed SLA (30–60 min instead of 10–15)

This alone can cut the required SKU count per store by 40–60% without losing catalog breadth for the customer.

---

2. Data Required

2.1 Transactional Data (most important — get 6–12 months minimum, at daily grain)

Data Point	Why It Matters
SKU–store–day sales (units, revenue, gross margin)	Core input for demand forecasting
Stockout events / out-of-stock duration per SKU–store	**Critical.** Without this, sales history is *censored* — a stocked-out SKU looks like "no demand" when it's actually "no stock"
Returns, wastage / write-offs	Essential for perishables — tells you what's overstocked relative to

	shelf life
Expiry dates at batch level	Needed for shelf-life-aware reorder policies

2.2 Product Master

- Category → sub-category → brand → SKU → pack size hierarchy
- Dimensions (L × W × H) or footprint per unit — needed to convert “space” into actual sqft/shelf-facing terms
- Cost price, MRP, margin, shelf life (days), storage type (ambient / chilled / frozen)
- A substitutability/complementarity tag (e.g., two 500ml coconut oil brands are substitutes; shampoo and conditioner are complements)

2.3 Store Master

- Store sqft, usable shelf/rack linear length, chiller/freezer capacity tracked separately
- Locality type (residential / office / student / mixed), average order value, order density per sqft, delivery radius, footfall/order trends
- Local demographic proxies (income band, family size, tier-1/2/3 city)

2.4 Customer / Demand Signal (often the most underused)

- **App search terms with zero results** — a direct “unmet demand” signal, gold for finding assortment gaps
- Product page views without add-to-cart, add-to-cart without purchase (funnel drop by SKU)
- Basket-level data for co-purchase / complementary analysis

2.5 Ops / Supply

- Replenishment lead time and frequency per store, vendor MOQs
- Current planogram / facing count, if one already exists

---

3. If the Data Doesn’t Exist Yet: Questions to Ask & Fallbacks

Ask your ops/product team directly:

- **“Do we log stockouts, or does a stocked-out SKU just disappear from the app with zero visibility?”** If no — fix this first. It’s the single highest-value data pipeline to build. Without it, you cannot separate “nobody wants this” from “we never had it in stock.”
- **“Do we capture zero-result searches and product page views?”** If not tracked in app analytics yet, this is usually a quick event-tracking addition and one of the highest-ROI instrumentation changes you can make.
- **“Is there a single product master with dimensions?”** Often exists partially. Missing dimensions can be approximated from vendor catalogs, or measured for the top 500 SKUs first (Pareto — don’t gold-plate the long tail).
- **“What do we do for a brand-new store with zero sales history?”** Use a **donor-store approach**: find the most similar existing store (by size, locality type, demographics) and copy its optimized assortment as a cold-start prior. Let 4–6 weeks of live sales correct it from there.

---

4. System Architecture: The Optimization Pipeline



```
└──────────────┬──────────────┘
               │
┌──────────────▼──────────────┐
│     Network Tiering         │  Core / regional hub / long-tail split
└──────────────┬──────────────┘
               │
┌──────────────▼──────────────┐
│ Inventory & Replenishment   │  Safety stock, reorder points, wastage control
└──────────────┬──────────────┘
               │
┌──────────────▼──────────────┐
│   Monitor & Retrain         │  Sell-through, stockouts, dead stock ──────┐
└─────────────────────────────────────────────────────────────────────────  │
               ▲                                                           │
               └──────────────────── feeds back into Data Foundation ──────┘
```

Each stage is solved for a small number of **store archetypes/clusters** (15–30), not for every individual store — this keeps the system computationally tractable and operationally manageable.

---

5. The Core Math: Shelf-Space Allocation Problem (SSAP)

This is a well-studied operations research problem. At its heart, it's a constrained knapsack / Mixed-Integer Programming (MIP) problem.

Decision variable: `x_i ∈ {0, 1}` — whether SKU *i* is stocked in store *s* (extend to integer `f_i` = number of facings for facing-level granularity)

Objective — maximize expected contribution profit:

```
maximize   Σ [ margin_i × D_i(f_i) − holding_cost_i(f_i) − wastage_cost_i(f_i) ] × x_i
```

Where `D_i(f_i)` is a **space-elasticity demand function**. Sales don't scale linearly with facings — they follow diminishing returns (a classic retail finding: doubling facings might give +20–30% sales, not +100%). Model this as a power/log-log curve:

```
D_i(f_i) = a_i × f_i ^ b_i        (0 < b_i < 1, diminishing returns)
```

fit from historical facing-change experiments, or approximated with a simple concave curve if no facing-variation history exists yet.

Constraints:

```
Σ space_i × x_i          ≤ Total_available_space_in_store      (the binding constraint)
Σ cost_i  × x_i          ≤ Inventory_capital_budget
x_i                      ≥ min_variety_per_subcategory   (preserve customer perception of choice —
                                                            e.g. keep ≥3 hair oil brands even if 2 dominate sales)
x_i                      ≤ max_share_per_subcategory     (don't let one brand hog 80% of a subcategory's space)
category_min% ≤ Σ space in category ≤ category_max%      (macro space plan by category role)
```

This is solvable at scale using a MIP solver (Google OR-Tools, Gurobi, or open-source CBC) run **per store-cluster**, not per individual store.

---

6. The Practical Cascade Before Full MIP

Before jumping to a full MIP, run this cheaper cascade first — it captures roughly 70% of the value for a fraction of the effort:

1. **ABC-XYZ Analysis**
   - ABC on revenue/margin contribution (A = top 20% of SKUs driving ~80% of sales)
   - XYZ on demand variability (X = stable demand, Z = erratic demand)
   - Combine into a 9-box grid: AX/AY SKUs are automatic keeps, CZ SKUs are automatic long-tail candidates
2. **Category Role Tagging** (manual + data-driven)
   - **Destination / traffic-driver** — staples like atta, milk, eggs. Keep deep even if margin is thin.
   - **Margin driver** — personal care, packaged snacks
   - **Convenience / impulse** — small footprint, near "checkout" placement
   - **Seasonal / occasional** — route to regional hub, not every store
3. Only run the full MIP **within each category** to fine-tune the SKU/size mix once roles and macro-space allocation are set.

7. Handling Substitution & Cannibalization

For a category like hair oil (5 brands × 5 sub-categories × 3 sizes = 75 SKUs):

Run **market basket / association analysis** (Apriori or FP-Growth) and a **cross-price/cross-stock elasticity** check:

> *When Brand A's 200ml is stocked out, does Brand B's 200ml sales spike (true substitution — low risk to cut Brand A) or does the customer simply not buy at all (true incremental demand — high risk to cut)?*

You get this from analyzing **natural experiments in your own stockout history** — no new data collection needed, just analysis of what already happened during past OOS periods.

Practical structure for a 75-SKU category:

- Keep the top 2 brands × 2–3 best-selling sub-categories × 2 sizes in **every store** (≈8–12 SKUs) — this typically covers 70–80% of category demand
- Keep 1 additional brand/size combo per **store cluster** based on local preference (some localities skew premium, others economy)
- Route the remaining 50+ SKUs to a regional hub or "extended catalog" — same app, slightly longer delivery window, fulfilled from a larger node

---

8. Network Tiering: The Biggest Structural Lever

Since delivery is fast anyway, define three tiers:

Tier	Location	Criteria	Typical SLA
Tier 1 — Core	Every dark store	High velocity, high frequency, low variability	10–15 min
Tier 2 — Regional Hub	1 hub per 5–10 dark stores	Medium velocity, replenished same-day into stores	25–40 min
Tier 3 — Extended / Long-tail	Larger fulfillment center	Low velocity, catalog-only, still visible in-app	1–2 hrs / next-day

This turns "75 SKUs in 1,200 sqft" into "12–15 core SKUs in-store + 60 SKUs one click away" — reconciling a small footprint with full catalog depth, and typically covering ~200–400 core SKUs total for a store this size.

---

9. Inventory & Replenishment Policy

For non-perishables: Standard **(s, S) reorder-point policy**

```
Safety stock = z × σ_demand × √(lead time)
```

where `z` is the service-level factor (e.g., z = 1.65 for ~95% service level) and `σ_demand` is demand variability.

For perishables: Use a **newsvendor model** — order quantity that balances the cost of understock (lost margin) against overstock (wastage before expiry):

```
Critical ratio = (price − cost) / (price − salvage_value)
```

Feed shelf-life directly into this — a 3-day shelf-life SKU needs a much tighter reorder cycle than a 6-month one, regardless of what the pure demand forecast says.

---

10. ML Techniques Mapped to Each Stage

Stage	Technique
Store clustering	K-means / hierarchical clustering on store attributes + sales-mix vectors
Demand forecasting	LightGBM / XGBoost with store, SKU, seasonality, promo features; or hierarchical forecasting with reconciliation across store → cluster → city
Unconstraining censored demand	Imputation using pre-stockout run-rate, or EM-style demand estimation methods used in revenue management
Space elasticity	Power-law / log-log regression fit on historical facing-change or comparable-store natural experiments
Substitution mapping	Apriori / FP-Growth basket analysis + stockout cross-elasticity
Assortment selection	MIP (OR-Tools / Gurobi) per cluster; simulated annealing or genetic algorithms if scale exceeds exact-solve capacity
Continuous SKU testing (new launches, marginal SKUs)	Contextual multi-armed bandits — trial a candidate SKU in a subset of stores, expand on success

---

11. Rollout Roadmap: Crawl → Walk → Run

Phase 1 — Weeks 1–4 Fix stockout logging if missing. Run ABC-XYZ analysis + manual category-role tagging. Ship a rules-based core assortment per store cluster. This alone usually captures most of the upside.

Phase 2 — Weeks 4–10 Build demand/elasticity models, formalize network tiering, automate the MIP per cluster, run it monthly.

Phase 3 — Ongoing Bandit-based testing for new SKUs, weekly model retraining, dashboards for sell-through/stockout/dead-stock, quarterly re-clustering as stores mature.

---

12. KPIs & Feedback Loop

Track continuously and feed results back into the data foundation for retraining:

- **Sell-through rate** per SKU per store
- **Days of inventory** on hand
- **Stockout %** (frequency and duration)
- **Dead stock value** (SKUs with near-zero velocity still occupying space)
- **Space productivity** (revenue/margin per sqft per category)
- **Basket completion rate** (did the customer find what they searched for)

---

13. Key Assumptions

Flag any of these that don’t hold true for your business, since they affect the design above:

1. You have (or can build within weeks) SKU–store–day sales and stockout data — everything downstream depends on this.
2. Store space is the binding constraint, not just inventory capital (both are modeled above, but space usually binds first at 800–1,500 sqft).
3. Delivery SLA flexibility exists for tiering — i.e., not every SKU needs to be physically within the 15-minute radius.
4. You’re optimizing for **contribution margin**, not pure revenue — worth confirming with finance, since staples (thin margin, high frequency) play a different strategic role than personal care (fat margin, lower frequency).

Document prepared as a system design framework for SKU rationalization and space optimization in quick-commerce dark stores.